

## Practical No.4

### Aim

To implement and compare two Pseudo-Random Number Generator (PRNG) algorithms: Linear Congruential Generator (LCG) and Blum Blum Shub (BBS) Generator. Additionally, perform statistical tests including uniformity test (Chi-Square test), scalability test (performance analysis), and consistency test (statistical analysis) to evaluate the quality and performance of the generated random numbers.

### Code

```
// Random Number Generation and Testing - TypeScript Implementation
// Implementation and comparison of two PRNG algorithms:
// 1. Linear Congruential Generator (LCG)
// 2. Blum Blum Shub (BBS) Generator

// Linear Congruential Generator (LCG)
// Formula:  $X(n+1) = (a * X(n) + c) \bmod m$ 
class LinearCongruentialGenerator {
    private m: number;
    private a: number;
    private c: number;
    private state: number;

    constructor(m: number = 2 ** 32, a: number = 16645, c: number = 1013904) {
        this.m = m;
        this.a = a;
        this.c = c;
        this.state = seed ?? Date.now();
    }

    // Generate next random number
    next(): number {
        this.state = (this.a * this.state + this.c) % this.m;
        return this.state;
    }

    // Generate next 32-bit random number
```

```
next32bit(): number {
    return this.next() & 0xFFFFFFFF;
}

// Generate random float in [0, 1)
nextFloat(): number {
    return this.next() / this.m;
}

// Generate a sequence of random numbers
generateSequence(count: number): number[] {
    const sequence: number[] = [];
    for (let i = 0; i < count; i++) {
        sequence.push(this.next32bit());
    }
    return sequence;
}

// Generate random number with specified number of bits
generateBits(numBits: number): number {
    let result = 0;
    const bitsPerCall = 32;
    const calls = Math.ceil(numBits / bitsPerCall);

    for (let i = 0; i < calls; i++) {
        result = (result << bitsPerCall) | this.next32bit();
    }

    // Mask to exact number of bits
    return result & ((1 << numBits) - 1);
}

// Blum Blum Shub (BBS) Generator
// Formula:  $X(n+1) = X(n)^2 \bmod n$ 
// where  $n = p * q$ , and  $p, q$  are primes congruent to 3 mod 4
class BlumBlumShub {
    private p: number;
    private q: number;
```

```
private n: number;
private state: number;

constructor(p: number = 3, q: number = 11, seed?: number) {
  // Verify p and q are Blum primes ( 3 mod 4)
  if (p % 4 !== 3 || q % 4 !== 3) {
    throw new Error('p and q must be primes congruent to 3 mod 4');
  }

  this.p = p;
  this.q = q;
  this.n = p * q;
  this.state = (seed ?? Date.now()) % this.n;

  // Ensure seed is coprime to n
  if (this.gcd(this.state, this.n) !== 1) {
    this.state = 2;
  }
}

private gcd(a: number, b: number): number {
  a = Math.abs(a);
  b = Math.abs(b);

  while (b !== 0) {
    [a, b] = [b, a % b];
  }
  return a;
}

// Generate next single random bit
nextBit(): number {
  this.state = (this.state * this.state) % this.n;
  return this.state & 1;
}

// Generate 32-bit random number
next32bit(): number {
  let result = 0;
```

```
    for (let i = 0; i < 32; i++) {
        result = (result << 1) | this.nextBit();
    }
    return result;
}

// Generate random float in [0, 1)
nextFloat(): number {
    return this.next32bit() / (2 ** 32);
}

// Generate a sequence of random numbers
generateSequence(count: number): number[] {
    const sequence: number[] = [];
    for (let i = 0; i < count; i++) {
        sequence.push(this.next32bit());
    }
    return sequence;
}

// Generate random number with specified number of bits
generateBits(numBits: number): number {
    let result = 0;
    for (let i = 0; i < numBits; i++) {
        result = (result << 1) | this.nextBit();
    }
    return result;
}
}

// Test 1: Uniformity Test (Chi-Square Test)
class UniformityTest {
    // Perform chi-square test for uniformity
    static chiSquareTest(numbers: number[], bins: number = 16, alpha: number) {
        const n = numbers.length;
        const expectedFreq = n / bins;

        // Count observed frequencies
        const counts = new Map<number, number>();
```

```
for (let i = 0; i < bins; i++) {
    counts.set(i, 0);
}

for (const num of numbers) {
    const bin = num % bins;
    counts.set(bin, (counts.get(bin) ?? 0) + 1);
}

// Calculate chi-square statistic
let chiSquare = 0;
for (let i = 0; i < bins; i++) {
    const observed = counts.get(i) ?? 0;
    chiSquare += ((observed - expectedFreq) ** 2) / expectedFreq;
}

// Degrees of freedom
const df = bins - 1;

// Critical value at alpha (approximate for common values)
const criticalValues: Record<number, Record<number, number>> = {
    0.05: { 15: 25.00, 7: 14.07, 3: 7.81 },
    0.01: { 15: 30.58, 7: 18.48, 3: 11.34 }
};

const criticalValue = criticalValues[alpha]?.[df];

// Determine result
let passed: boolean;
if (criticalValue !== undefined) {
    passed = chiSquare < criticalValue;
} else {
    // For other values, use rough approximation
    passed = chiSquare < df + 3 * Math.sqrt(2 * df);
}

return {
    chiSquare,
    degreesOfFreedom: df,
```

```
        criticalValue,
        alpha,
        passed,
        interpretation: passed ? 'UNIFORM' : 'NOT UNIFORM'
    };
}

// Analyze frequency distribution
static frequencyAnalysis(numbers: number[], bins: number = 16): FrequencyAnalysis {
    const counts = new Map<number, number>();
    for (let i = 0; i < bins; i++) {
        counts.set(i, 0);
    }

    for (const num of numbers) {
        const bin = num % bins;
        counts.set(bin, (counts.get(bin) ?? 0) + 1);
    }

    const expected = numbers.length / bins;

    const frequencies: Record<string, number> = {};
    for (let i = 0; i < bins; i++) {
        frequencies[`bin_${i}`] = counts.get(i) ?? 0;
    }

    // Calculate statistics
    const observedValues = Array.from(counts.values());
    const variance = observedValues.reduce((sum, x) => sum + (x - expected)2);
    const stdDeviation = Math.sqrt(variance);

    return {
        frequencies,
        expectedFrequency: expected,
        variance,
        stdDeviation
    };
}
```

```
// Test 2: Scalability Test (Performance Analysis)
class ScalabilityTest {
    // Measure generation time for different bit sizes
    static measurePerformance(generator: LCG | BBS, numBitsList: number[],
        const results: Record<string, PerformanceMetric> = {});

    for (const numBits of numBitsList) {
        const times: number[] = [];

        for (let i = 0; i < iterations; i++) {
            const start = performance.now();
            generator.generateBits(numBits);
            const end = performance.now();
            times.push(end - start);
        }

        const avgTime = times.reduce((a, b) => a + b, 0) / times.length;
        const variance = times.reduce((sum, t) => sum + (t - avgTime) ** 2, 0);
        const stdTime = Math.sqrt(variance);
        const throughput = avgTime > 0 ? 1 / (avgTime / 1000) : 0;

        results[`_${numBits}_bits`] = {
            avgTimeNs: avgTime * 1_000_000,
            stdTimeNs: stdTime * 1_000_000,
            throughputPerSec: throughput
        };
    }

    return results;
}

// Analyze time complexity for generating sequences
static timeComplexityAnalysis(generator: LCG | BBS, sizes: number[]): T {
    const results: Record<number, TimeMetric> = {};

    for (const size of sizes) {
        const start = performance.now();
        generator.generateSequence(size);
```

```
    const end = performance.now();

    const elapsed = (end - start) / 1000; // Convert to seconds

    results[size] = {
      timeSeconds: elapsed,
      timePerNumberNs: (elapsed / size) * 1_000_000_000
    };
  }

  return results;
}

// Test 3: Consistency Test (Statistical Analysis)
class ConsistencyTest {
  // Test mean and variance
  static meanVarianceTest(numbers: number[]): MeanVarianceResult {
    const n = numbers.length;
    const mean = numbers.reduce((sum, x) => sum + x, 0) / n;
    const variance = numbers.reduce((sum, x) => sum + (x - mean) ** 2, 0);
    const stdDev = Math.sqrt(variance);

    return {
      mean,
      variance,
      stdDev,
      count: n
    };
  }

  // Test autocorrelation
  static autocorrelationTest(numbers: number[], lag: number = 1): number {
    const n = numbers.length;
    const mean = numbers.reduce((sum, x) => sum + x, 0) / n;

    let numerator = 0;
    let denominator = 0;
```

```
    for (let i = 0; i < n - lag; i++) {
        numerator += (numbers[i] - mean) * (numbers[i + lag] - mean);
    }

    for (let i = 0; i < n; i++) {
        denominator += (numbers[i] - mean) ** 2;
    }

    return numerator / denominator;
}

// Type aliases for clarity
type LCG = LinearCongruentialGenerator;
type BBS = BlumBlumShub;

// Result interfaces
interface ChiSquareResult {
    chiSquare: number;
    degreesOfFreedom: number;
    criticalValue?: number;
    alpha: number;
    passed: boolean;
    interpretation: string;
}

interface FrequencyAnalysisResult {
    frequencies: Record<string, number>;
    expectedFrequency: number;
    variance: number;
    stdDeviation: number;
}

interface PerformanceMetric {
    avgTimeNs: number;
    stdTimeNs: number;
    throughputPerSec: number;
}
```

```
interface PerformanceResult {
  [key: string]: PerformanceMetric;
}

interface TimeMetric {
  timeSeconds: number;
  timePerNumberNs: number;
}

interface TimeComplexityResult {
  [key: number]: TimeMetric;
}

interface MeanVarianceResult {
  mean: number;
  variance: number;
  stdDev: number;
  count: number;
}

// Demonstration function
function demonstration(): void {
  console.log('=' .repeat(70));
  console.log('RANDOM NUMBER GENERATION COMPARISON');
  console.log('=' .repeat(70));

  // Initialize generators
  const lcg = new LinearCongruentialGenerator();
  const bbs = new BlumBlumShub();

  const sampleSizes = [100, 1000, 10000];

  for (const size of sampleSizes) {
    console.log(`\n--- Sample Size: ${size} ---`);

    // Generate sequences
    const lcgSequence = lcg.generateSequence(size);
    const bbsSequence = bbs.generateSequence(size);
```

```

// Uniformity test
console.log('\nUniformity Test (Chi-Square):');
const lcgChi = UniformityTest.chiSquareTest(lcgSequence);
const bbsChi = UniformityTest.chiSquareTest(bbsSequence);

console.log(` LCG: \u03c7\u00b2 = ${lcgChi.chiSquare.toFixed(2)}, ${`
console.log(` BBS: \u03c7\u00b2 = ${bbsChi.chiSquare.toFixed(2)}, ${`

// Mean and variance
console.log('\nStatistical Properties:');
const lcgStats = ConsistencyTest.meanVarianceTest(lcgSequence);
const bbsStats = ConsistencyTest.meanVarianceTest(bbsSequence);

console.log(` LCG: mean = ${lcgStats.mean.toFixed(2)}, std = ${lcgSt
console.log(` BBS: mean = ${bbsStats.mean.toFixed(2)}, std = ${bbsSt

// Performance
console.log('\nPerformance:');
const perf = ScalabilityTest.measurePerformance(lcg, [32], 100);
const lcgPerf = perf['32_bits'];
console.log(` LCG: ${lcgPerf.avgTimeNs.toFixed(2)} ns per 32-bit num

const bbsPerfResult = ScalabilityTest.measurePerformance(bbs, [32], 1
const bbsPerf = bbsPerfResult['32_bits'];
console.log(` BBS: ${bbsPerf.avgTimeNs.toFixed(2)} ns per 32-bit num
}

console.log('\n' + '='.repeat(70));
console.log('SUMMARY');
console.log('='.repeat(70));
console.log(`
1. LINEAR CONGRUENTIAL GENERATOR (LCG):
  - Fast and efficient
  - Good uniformity for well-chosen parameters
  - Not cryptographically secure
  - Period limited by modulus

2. BLUM BLUM SHUB (BBS):
  - Slower due to large integer operations

```

- Cryptographically secure under certain assumptions
- Good statistical properties
- Requires careful parameter selection

Recommendations:

- Use LCG for simulations and non-critical applications
- Use BBS for cryptographic applications (with larger primes)
- Always test PRNG properties for your specific use case

```
`);  
}  
  
// Main function  
function main(): void {  
    console.log('Random Number Generation and Testing Program');  
    console.log('=====\n');  
  
    demonstration();  
}  
  
// Run if executed directly  
main();
```

## Output

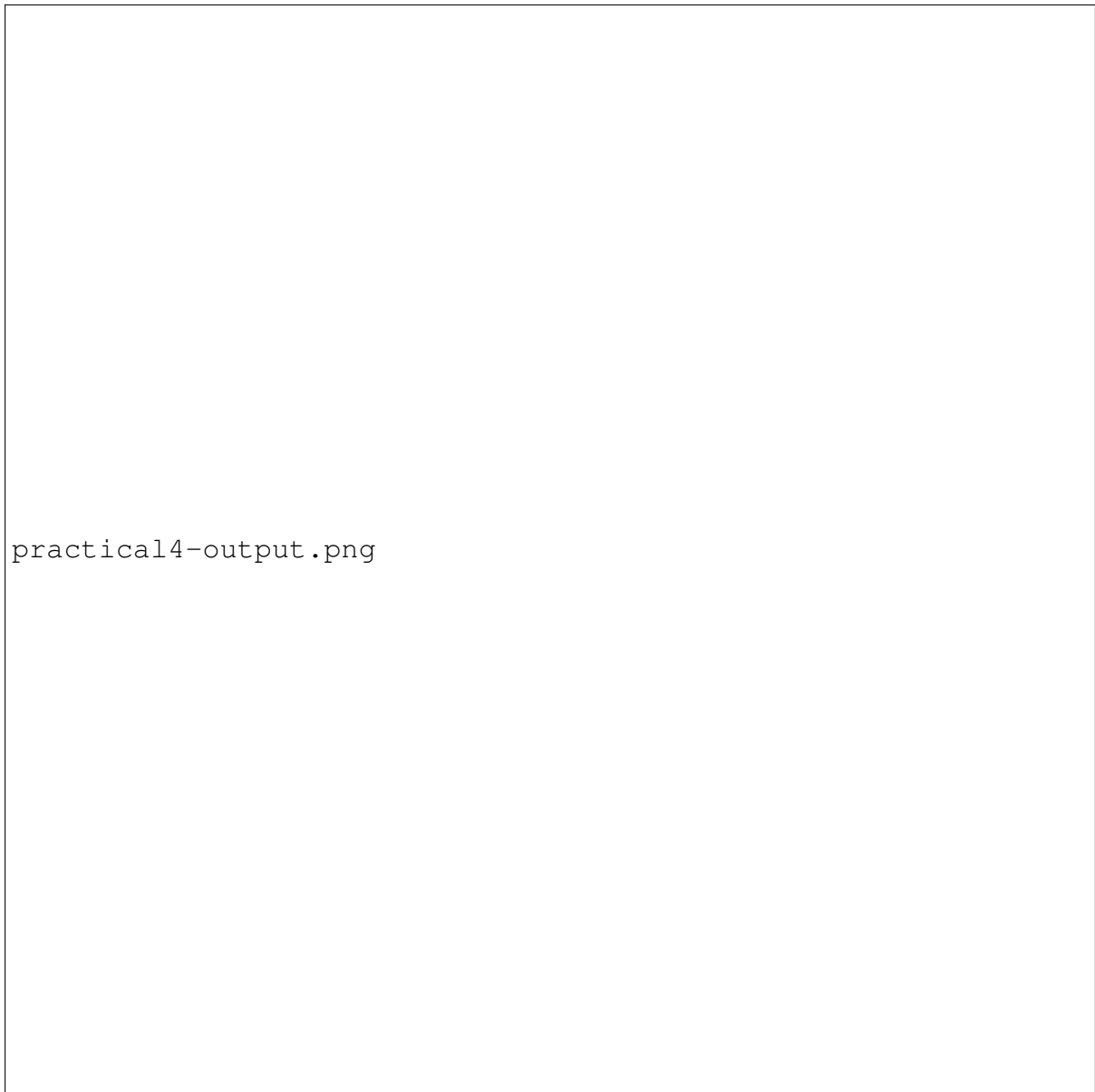


Figure 1: Output of Random Number Generation and Testing Program